# ML Pipeline Foundation Report

*M5 Retail Demand Forecasting — Problem Statement 11*

*Stage 1: data loading, feature engineering, leakage-safe backtesting*

*NPN AIA Hackathon — St. Joseph's College of Engineering*

---

*Stage 1 of the forecasting build. Generated 2026-08-14 from an actual pipeline run (16.5s).*

> ***No model has been trained in this stage.*** *No hyperparameters were tuned, no hurdle model was built, no submission was created, and no claim is made about the team's LightGBM+Tweedie benchmark (RMSE 2.0324 / MAE 1.0869). This report documents only the machinery that a model will later be trained on.*

---

## 1. What we built, in one paragraph

We built the scaffolding for forecasting 28 days of daily unit sales for all 30,490 store-item series. That scaffolding has four parts: a **data loader** that reads the raw files into compact matrices, a **feature engineering layer** that turns history into model inputs without ever peeking at the future, a **backtesting framework** that recreates the real 28-day forecasting task on a stretch of history where we already know the answers, and a **check suite** that tries to prove the whole thing wrong. All 46 checks currently pass.

### Project structure

```
pipeline/
    config.py             paths, dataset constants, backtest origins
    data_loader.py        raw CSVs -> wide matrices (read-only)
    features.py           feature engineering, groups A-G
    backtest.py           train / validation / future frame assembly
    metrics.py            RMSE, MAE, WAPE, bias
    validation_checks.py  correctness + empirical leakage tests
    report_pdf.py         markdown -> PDF renderer
scripts/
    01_foundation_check.py        runs everything, writes the results JSON
    02_build_foundation_report.py builds this report from that JSON
artifacts/    check results, feature summary, inspectable sample
models/       (empty) trained model files, next stage
experiments/  (empty) ablation configs and results, next stage
predictions/  (empty) forecast outputs, final stage
reports/      this report
```

## 2. The data we are working from

The pipeline reads the original files in `raw_dataset/` directly, **read-only**. Nothing in `raw_dataset/` or `processed_dataset/` was modified, and `sales_long_full.parquet` was not touched.

| Property | Value |
|---|---|
| Store-item series | 30,490 |
| Days of history | 1,941 (2011-01-29 to 2016-05-22) |
| Calendar days available | 1,969 (runs to 2016-06-19) |
| Price weeks | 282 |
| Sales matrix in memory | 118.4 MB |
| Price matrix in memory | 34.4 MB |
| Load time | 13.6s |

> ***Why not the 59-million-row processed table?*** `processed_dataset/sales_long_full.parquet` *holds the same information but as 59,181,090 separate rows, which needs several GB to work with. This machine has about 5.7 GB free. Kept in its natural rectangular shape (30,490 series x 1,941 days) the same data fits in 118.4 MB, and every feature becomes a fast array slice instead of a grouped scan over 59 million rows. The processed table is left exactly as it was; we cross-check our totals against the values already verified from it.*

The loader refuses to proceed unless the data reproduces the totals that were independently verified in the earlier review stage:

| Integrity check | Expected | Result |
|---|---|---|
| Total units sold | 66,927,173 | matched |
| Zero-sales cells | 40,241,819 (68.00%) | matched |
| Maximum single-day sale | 763 | matched |
| Negative sales values | 0 | matched |

## 3. What leakage means, and why this stage is mostly about preventing it

> ***Feature leakage*** *is when information from the future accidentally ends up in the data a model learns from. The model then looks brilliant during testing and falls apart in reality, because at the real moment of forecasting that information does not exist yet.*

A concrete example for this project. Suppose we want to predict sales on 25 May. A feature like "average sales over the last 7 days" sounds harmless — but if we compute it *relative to 25 May*, it uses 18-24 May. If we are actually standing on 1 May making a 28-day forecast, we do not know any of those values yet. Using them is leakage.

This is not a hypothetical risk. It is the single most common way a forecasting project quietly fools itself, and both the EDA report and the final approach document flagged it as the biggest correctness risk in the project. So the pipeline is built around one rule.

### The rule: fixed origin, two kinds of feature

Pick a day **T**, the *forecast origin* — the last day whose sales we know. We must predict days T+1 through T+28 all at once, standing at T. Every feature is then one of exactly two kinds:

| Kind | Built from | Behaviour across the 28 days |
|---|---|---|
| **Origin-relative** | sales history up to and including day T | **Constant.** The same value is used for all 28 forecast days. |
| **Target-day** | the calendar and the price file, which are published ahead of time | **Varies** per forecast day, legitimately. |

Everything derived from past sales is origin-relative, because on day T we genuinely do not know day T+5's sales. Everything derived from the calendar or prices is target-day, because this dataset really does supply those for the forecast window.

### What the model is allowed to see, and what it must never see

| Information | Allowed? | Why |
|---|---|---|
| Sales on or before day T | **YES** | Already observed at forecast time |
| Sales after day T | **NEVER** | This is the answer we are being asked for |
| Weekday, month, year of a forecast day | **YES** | The calendar is deterministic |
| Holiday / event name on a forecast day | **YES** | `calendar.csv` covers all 28 future days |
| SNAP flag on a forecast day | **YES** | Benefit schedules are published in advance |

| Information | Allowed? | Why |
|---|---|---|
| Selling price on a forecast day | **YES** | `sell_prices.csv` covers the forecast weeks |
| Any aggregate computed across the forecast window | **NEVER** | Would smuggle future sales in indirectly |

***SNAP*** *is the Supplemental Nutrition Assistance Program, a US food-assistance benefit. The dataset flags, per state per day, whether the benefit was usable. Each series is matched to its* ***own*** *state's flag — a California store reads* `snap_CA` *— because using a blended flag would blur a signal the EDA found to be worth +12.7% in mean sales overall and +17.3% within FOODS.*

## 4. How the 28-day backtest works

We cannot score anything on the real forecast window, because nobody has those sales — predicting them is the whole task. So we rewind: pretend an earlier day was "today", forecast the 28 days after it, and compare against sales we already have on record.

***Why random train/test splitting would be wrong.*** *A random split would let the model train on 10 May while being tested on 30 April — learning from the future to explain the past. The score would look excellent and mean nothing. Time-series validation must always cut on time.*

### The three blocks of data, kept strictly apart

| Block | Days | Dates | Role |
|---|---|---|---|
| **TRAINING** | d_1 .. d_1913 | 2011-01-29 .. 2016-04-24 | Everything the model may learn from |
| **VALIDATION** | d_1914 .. d_1941 | 2016-04-25 .. 2016-05-22 | Real observed sales. Scored against, never trained on, never used to build a feature |
| **FINAL FORECAST** | d_1942 .. d_1969 | 2016-05-23 .. 2016-06-19 | Genuinely unknown. No sales for these days exist in any file |

The validation origin is **d_1913 (2016-04-24)**. That day was chosen deliberately: the 28 days after it (d_1914 .. d_1941) are exactly the block that exists in `sales_train_evaluation.csv` but not in `sales_train_validation.csv`. They are real observed sales, so we can score against them, and they reproduce the shape of the real task precisely — 28 days ahead, from a fixed origin, for every series at once.

### A subtlety that is easy to get wrong

It is not enough for training *targets* to sit before the cutoff. Training *features* must also be buildable from before the cutoff. So every training origin satisfies `origin + 28 <= validation origin`. The training origins used in this verification run were:

| Training origin | Date | Its 28-day target block ends |
|---|---|---|
| d_1745 | 2015-11-08 | d_1773 |
| d_1773 | 2015-12-06 | d_1801 |
| d_1801 | 2016-01-03 | d_1829 |
| d_1829 | 2016-01-31 | d_1857 |
| d_1857 | 2016-02-28 | d_1885 |
| d_1885 | 2016-03-27 | d_1913 |

The latest of those target blocks ends at d_1913, one day before the validation window opens at d_1914. The framework asserts this rather than assuming it, and raises an error if it is ever violated.

## 5. The features

**32 features across 7 groups.** Every one is labelled below as origin-relative (constant over the 28 days) or target-day (varies).

### Group A — Calendar (target-day)

Why it exists: the EDA measured a +31.1% weekend effect and found individual named holidays moving in opposite directions (Christmas -99.95%, Labor Day +27.5%). A single "is it a holiday" flag would average that away to nearly nothing, so the specific event identity is kept.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| wday | 0.0 | 1.0 | 7.0 | 4.0 |
| month | 0.0 | 4.0 | 5.0 | 4.7857 |
| year | 0.0 | 2016.0 | 2016.0 | 2016.0 |
| is_weekend | 0.0 | 0.0 | 1.0 | 0.2857 |
| event_name_1 | 0.0 | 0.0 | 22.0 | 2.2143 |
| event_type_1 | 0.0 | 0.0 | 3.0 | 0.2857 |
| event_name_2 | 0.0 | 0.0 | 0.0 | 0.0 |
| event_type_2 | 0.0 | 0.0 | 0.0 | 0.0 |
| snap | 0.0 | 0.0 | 1.0 | 0.3571 |

### Group B — Historical Demand (origin-relative)

Why it exists: recent demand is the strongest predictor in the dataset. The EDA measured rolling_mean_7 at r=0.820 and rolling_mean_28 at r=0.807 against same-day sales, higher than any single lag.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| lag_1 | 0.0 | 0.0 | 130.0 | 1.6332 |
| lag_7 | 0.0 | 0.0 | 98.0 | 1.2482 |
| lag_14 | 0.0 | 0.0 | 122.0 | 1.3915 |
| lag_28 | 0.0 | 0.0 | 61.0 | 1.1821 |
| rolling_mean_7 | 0.0 | 0.0 | 108.7143 | 1.3366 |
| rolling_mean_28 | 0.0 | 0.0 | 110.3571 | 1.3864 |
| rolling_std_7 | 0.0 | 0.0 | 36.4641 | 1.0241 |
| rolling_std_28 | 0.0 | 0.0 | 39.2032 | 1.2366 |

### Group C — Recency (origin-relative)

Why it exists: the cleanest relationship found anywhere in the EDA. The chance of selling today falls from 65.2% if the item sold yesterday to 0.6% after 29+ dry days.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| days_since_last_sale | 0.0 | 0.0 | 1662.0 | 7.0681 |
| zero_streak_length | 0.0 | 0.0 | 1662.0 | 7.0681 |
| days_since_first_sale | 0.0 | 67.0 | 1912.0 | 1505.8057 |

### Group D — Listing (mixed)

Why it exists: many early zeros are not weak demand, they are "this product was not on the shelf yet". Section 7 shows this is measurable and near-absolute.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| days_since_first_listing | 0.0 | 72.0 | 1940.0 | 1523.1083 |
| pre_listing | 0.0 | 0.0 | 0.0 | 0.0 |

## Group E — Price (mixed)

Why it exists: price is one of the few genuinely forward-known variables here. Raw price is dominated by cross-item scale ($30 hobby item vs $1 food item), so price relative to the item's own recent average is included alongside it.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| sell_price | 0.0 | 0.1 | 33.72 | 4.4821 |
| recent_avg_price | 0.0 | 0.2 | 29.97 | 4.4813 |
| price_rel_to_recent_avg | 0.0 | 0.0915 | 2.8 | 1.0004 |
| price_is_missing | 0.0 | 0.0 | 0.0 | 0.0 |

## Group F — Hierarchy (static)

Why it exists: behaviour differs sharply across the hierarchy — zero-sales rates range from 58.6% in FOODS_3 to 88.4% in HOBBIES_2.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| item_id | 0.0 | 0.0 | 3048.0 | 1524.0 |
| dept_id | 0.0 | 0.0 | 6.0 | 3.161 |
| cat_id | 0.0 | 0.0 | 2.0 | 0.8721 |
| store_id | 0.0 | 0.0 | 9.0 | 4.5 |
| state_id | 0.0 | 0.0 | 2.0 | 0.9 |

## Group G — Horizon (target-day)

Why it exists: predicting 1 day ahead and 28 days ahead are different problems, and a direct multi-horizon model needs to know which one it is being asked for.

| Feature | Missing % | Min | Max | Mean |
|---|---|---|---|---|
| horizon | 0.0 | 1.0 | 28.0 | 14.5 |

## Exactly how the lag and rolling features are defined

`lag_k` = sales on the day k days before the **first forecast day**. With origin T, the first forecast day is T+1, so:

| Feature | Day it reads | Safe for all 28 horizon days? |
|---|---|---|
| lag_1 | T | Yes — T is the last day we know |
| lag_7 | T-6 | Yes |
| lag_14 | T-13 | Yes |
| lag_28 | T-27 | Yes |
| rolling_mean_7 / rolling_std_7 | mean/std over T-6 .. T | Yes |

| Feature | Day it reads | Safe for all 28 horizon days? |
|---|---|---|
| `rolling_mean_28` / `rolling_std_28` | mean/std over T-27 .. T | Yes |

All six windows end at T. None of them can reach into the forecast period, for any of the 28 days, which is what makes them usable in a direct multi-horizon setup.

### What we deliberately did NOT do to the data

- Sales were **not** smoothed.
- Zero-sales rows were **not** removed.
- Zeros were **not** replaced with missing values.
- No zero was assumed to be a stockout. The dataset has no inventory field, so that cannot be known.
- No suspected-stockout rows were dropped.
- No promotion labels were invented. The dataset has no promotion field.
- Missing prices were left as missing, never imputed. LightGBM handles them natively, and the missingness is itself informative.

The model will learn from the original observations exactly as recorded.

## 6. Validation checks

**46 of 46 checks pass.** The full machine-readable output is in `artifacts/foundation_checks.json`.

### The leakage test, done empirically rather than asserted

Writing "this feature is safe" in a comment proves nothing. So the pipeline proves it by experiment:

- Build the feature frame normally at the validation origin.
- Take a copy of the sales matrix and overwrite **every day after the origin** with an absurd value (9999 units).
- Rebuild the exact same feature frame from the corrupted data.
- Compare. If any feature value moved, that feature was reading the future.

**Result: all 32 features are bit-for-bit identical between the clean and corrupted runs.** A companion check confirms the target column *did* change, which proves the corruption actually reached the data and the test was meaningful rather than vacuous.

The mirror-image test matters just as much: corrupting future **prices** *should* change the price features, because prices for the forecast window are legitimately known. That check passes too — so we are neither leaking what we must not use, nor discarding what we are entitled to use.

### All checks by area

| Area | Checks | What is verified |
|---|---|---|
| Source integrity | 9 | Row/day counts, total units, zero count, max value, no negatives, zeros not silently converted to NaN |
| Calendar alignment | 2 | Six anchor day-index/date pairs exact; calendar extends 28 days past the sales |
| Frame structure | 5 | Row count, no duplicate (series, day) pairs, exactly 28 distinct target days, every series present on every day, horizon values 1..28 |
| Feature sanity | 14 | Non-negative demand features, recency in range, SNAP binary and state-matched, prices positive, target still raw integers |
| Target correctness | 1 | 500 random rows spot-checked back against `sales_train_evaluation.csv` |

| Area | Checks | What is verified |
|---|---|---|
| Leakage | 4 | Future-sales corruption test, corruption-applied counter-check, future-price usability, price/demand independence |
| Train/validation separation | 3 | Training targets strictly precede validation; origins at least 28 days back; no duplicate training rows |
| Listing behaviour | 3 | Feature activates at early origins; pre-listing rows have no sales; redundancy check |
| Future frame | 4 | Correct row count, no target attached, calendar+SNAP present, price coverage |
| Metric pipeline | 1 | Metrics run over the full 853,720-prediction window |

### Row-count arithmetic, confirmed

| Frame | Rows | Check |
|---|---|---|
| Validation | 853,720 | 30,490 series x 28 days |
| Future forecast | 853,720 | 30,490 series x 28 days |
| Training (6 origins, 2,000-series sample) | 336,000 | 2,000 x 28 x 6 |

## 7. A finding that came out of building this

The listing-aware features were probed across four origins to check they actually do something. They produced a result stronger than the EDA had established:

| Origin | Date | Rows flagged pre-listing | Mean sales on those rows | Zero-sales rate on those rows | Zero-sales rate on listed rows |
|---|---|---|---|---|---|
| d_201 | 2011-08-17 | 47.84% | 0.0 | 100.0% | 56.96% |
| d_701 | 2012-12-29 | 27.68% | 0.0 | 100.0% | 59.79% |
| d_1401 | 2014-11-29 | 3.39% | 0.0 | 100.0% | 64.51% |
| d_1913 | 2016-04-24 | 0.0% | — | — | 54.44% |

**Rows flagged as pre-listing have a 100.00% zero-sales rate and a mean of exactly 0.0 units.** Not approximately — every single one. The flag is derived purely from `sell_prices.csv`, and the sales come from `sales_train_evaluation.csv`, so this is two independent files agreeing, not circular reasoning.

Two consequences worth carrying into the modelling stage:

- At early origins nearly half the panel is structurally zero (47.84% at d_201). Training on those rows as though they were ordinary weak demand will pull the model toward predicting zero. They are closer to "not applicable" than to "no demand".
- At the validation origin and at the real forecast origin, **0%** of rows are pre-listing and 100% have a known price. So this feature contributes nothing at prediction time for this particular horizon. It matters for how we build the **training set**, not for the final forecast. That is a meaningful limitation on how much the "listing-aware" idea can be expected to move the final score, and it is better to know now than after building a novelty story around it.

## 8. Problems encountered

### The leakage test failed on first run — and was right to

On the first full run, `rolling_std_28` came back as changed by the corruption test, which would mean a feature was reading future sales. It was investigated before anything was altered.

The input slices fed to the calculation were **byte-identical** between the clean and corrupted runs, so no future data was being read. The differences were at most 3.8e-06 in absolute terms and 5.1e-07 relative — roughly four times float32 machine epsilon. The cause: pandas returned the sales matrix in Fortran (column-major) order, while the test's copy was C (row-major) order, and NumPy's pairwise summation groups values according to memory layout. Same numbers, different addition order, last-bit differences.

Rather than weaken the test to a tolerance — which would have blunted the one check most likely to catch a genuine future-data bug — the root cause was fixed:

- the sales matrix is now stored C-contiguous (which also matches our dominant access pattern of whole-row slices per series);
- rolling means and standard deviations accumulate in float64 before being narrowed to float32.

Exact bit-equality now holds and the test remains strict.

### Two feature pairs turned out to be redundant

Both were built because the stage specification asked for them, and both are reported rather than quietly dropped:

- `days_since_last_sale` and `zero_streak_length` are **the same number** at a fixed origin. If the last sale was 3 days ago then there are exactly 3 consecutive zero days ending at the origin. Verified identical across all 853,720 validation rows.
- `pre_listing` and `price_is_missing` were **identical at every origin tested**. Pre-listing is defined from the first priced day, so for the leading block the two coincide exactly.

One of each pair should be dropped before training. Keeping both adds compute and splits feature-importance between duplicate columns, which makes the ablation study harder to read.

### Features that are inert at the forecast origin

`pre_listing` and `price_is_missing` are constant zero at the validation origin, and `year` is constant 2016 across the validation window. Constant columns carry no information for a tree model at prediction time. They are retained because they are informative in training rows drawn from earlier origins, but this is worth knowing before reading anything into their feature importances.

## 9. Metric pipeline smoke test

> ***These are not model results.*** *They are two trivial arithmetic rules with no fitting of any kind, run purely to prove the metric code works on a real 853,720-row window. They are **not** baselines, and they must **not** be compared with the team's LightGBM+Tweedie benchmark.*

| Rule | RMSE | MAE | WAPE | Bias |
|---|---|---|---|---|
| Predict 0 for everything | 3.9161 | 1.4428 | 1.0000 | -1.4428 |
| Repeat each series' own 28-day average | 2.2430 | 1.0657 | 0.7386 | -0.0564 |

The metric code runs correctly over all 853,720 predictions. One thing these numbers illustrate is why MAE alone is a poor guide on this dataset: predicting zero everywhere achieves an MAE of 1.4428 while explaining none of the demand at all, which the WAPE of 1.0000 makes visible. Both RMSE and MAE are reported throughout this project, alongside WAPE, for that reason.

## 10. Where this leaves us

### Ready

- Data loading, verified against known totals
- 32 features across 7 groups, all leakage-tested
- Fixed-origin 28-day backtest with an enforced train/validation separation

- Metrics (RMSE, MAE, WAPE, bias), including per-group breakdowns
- Future-horizon frame for d_1942..d_1969, with calendar, SNAP, events and prices present for 100.0% of rows and no target attached
- LightGBM 4.7.0 installed; `requirements.txt` pins the full environment

### Decisions that need making before Model 1

- **How many training origins, and how far back.** This run used 6 origins at a 28-day stride purely to verify the mechanics. More origins means more training data but also more memory; the full build at 30,490 series is about 853,720 rows per origin.
- **Whether to drop the redundant feature in each pair** identified in Section 8. Recommended: yes, before the ablation study.
- **Whether to exclude pre-listing rows from training.** They are structurally zero, and including roughly half a panel of guaranteed zeros at early origins will bias the model. This is a real modelling decision with evidence behind it now, and it should be tested both ways rather than assumed.
- **The evaluation metric for the hackathon** is still unconfirmed. RMSE and MAE are computed because the team's benchmark is quoted in them.

### Nothing is blocking Model 0 / Model 1

The foundation runs end to end in 16.5s and all 46 checks pass. The next stage can build a naive baseline and a global LightGBM model on top of this without further groundwork.

---

*Generated by `scripts/02_build_foundation_report.py` from `artifacts/foundation_checks.json`. Every figure in this report is read from that file, which was written by an actual pipeline run — no number here was entered by hand. No model was trained in this stage.*